

RetroArch Gameplay Automation Project: Research and Roadmap Continuation

Overview

The **gameplay-automation-retroarch** repository provides a clean, modular framework for controlling RetroArch externally. An audit document outlines completed phases and a recommended roadmap for future development. As of June 2026, the project has completed phases 1–8, which established a deterministic scheduler, a YAML-driven macro runner, expanded controller support, macro quality-of-life improvements, a profile library, safety features, screenshot capture using `mss`, and read-only image diagnostics using Pillow ¹. The next recommended step is **Phase 9: named screen regions** ¹.

This report summarizes the current project status, outlines the requirements for the remaining phases, and provides practical guidance to implement Phase 9. It also includes research on Python libraries (`mss` and `Pillow`) relevant to screenshot capture and image cropping.

Current Project Status

The audit document lists completed phases and their purposes:

Phase	Purpose / Key Features	Evidence
1	Create modular repository layout (<code>src/core</code> , <code>src/input</code> , <code>src/scenarios</code>) ² .	Completed.
2	YAML-driven scripted playback and legacy sequence support ³ .	Completed.
3	Expanded controller actions: face buttons, D-pad, menu/select aliases, shoulders/triggers, stick clicks, analog directions ⁴ .	Completed.
4	Macro quality-of-life: named macros, repeats, chords, waits, recursion checks ⁵ .	Completed.
5	Profile library, profile discovery, dry-run and validation commands ⁶ .	Completed.
6	Safety/reliability layer: countdown, max runtime, Ctrl-C safe stop, button release, logging statuses ⁷ .	Completed.
7	Screenshot/perception foundation using <code>mss</code> ⁸ .	Completed.
8	Read-only image diagnostics with Pillow (<code>analyze_image.py</code> , crop/compare) ⁹ .	Completed.

The audit emphasises keeping all autonomy disabled until perception and diagnostics are robust ¹⁰ . It also notes a local Git stash containing files like **Downloads/RetroArch bot creation.html** and **inline-styles.css** and recommends leaving the stash untouched to avoid reintroducing README conflicts ¹¹ .

Remaining Phases (9–20)

The roadmap proposes the following remaining phases ¹ :

1. **Phase 9: Named screen regions** – define reusable regions in config and implement commands to list, capture and analyze them. Acceptance criteria include validation of bounds, name listing, region capture to PNG/JSON, and backward-compatible coordinate overrides. OCR and decision logic are explicitly excluded ¹ .
2. **Phase 10: Region catalogs per emulator/game layout** – support multiple sets of region definitions (e.g., for GBA 4:3, RetroArch menu) ¹² .
3. **Phase 11: Diagnostic overlays** – draw region boxes and labels on saved screenshots ¹³ .
4. **Phase 12: Baseline snapshots and drift checks** – save approved baseline screenshots and compare current images to detect drift ¹⁴ .
5. **Phase 13: Threshold-only screen checks** – define simple numerical metrics (e.g., percentage of changed pixels) that must pass before proceeding ¹⁵ .
6. **Phase 14: Template matching foundation** – detect small user-supplied UI images within named regions using OpenCV template matching ¹⁶ .
7. **Phase 15: Conditional wait diagnostics** – wait for a visual condition to be met within a timeout; still diagnostic only ¹⁷ .
8. **Phase 16: Guarded action gates** – allow a single declared action after verifying a condition; introduces the first bridge from vision into action ¹⁸ .
9. **Phase 17: State machine runner** – formalize macros as state machines with explicit transitions, timeouts and failure paths ¹⁹ .
10. **Phase 18: Save-state and recovery integration** – coordinate emulator save/load workflows for offline testing ²⁰ .
11. **Phase 19: OCR experiments** – test OCR in a region-scoped, diagnostic-only manner ²¹ .
12. **Phase 20: Autonomous route experiments** – offline-only limited automation with strict safety controls ²² .

External Research: Screenshot Regions and Image Cropping

Capturing Screen Regions with `mss`

The `mss` library is a cross-platform screenshot tool that supports capturing monitors or arbitrary rectangles. A blog post (January 2026) demonstrates how to use `mss` for region capture ²³ :

```
import mss
from mss import tools

with mss.mss() as sct:
    # Define a region dictionary
    region = {
```

```

    'left': 100, # X coordinate
    'top': 100,  # Y coordinate
    'width': 500, # width in pixels
    'height': 500 # height in pixels
}
screenshot = sct.grab(region)
# Save the region as PNG
tools.to_png(screenshot.rgb, screenshot.size, output='region.png')

```

Key points:

- `sct.monitors[0]` represents all monitors combined; indices start at 1 for individual monitors ²⁴.
- A region is defined by a dictionary containing `left`, `top`, `width` and `height` ²⁵.
- `mss.tools.to_png()` converts the raw pixel data to a PNG file ²³.

Cropping and Drawing Overlays with Pillow

The Pillow (`PIL`) library provides image manipulation functions, including cropping and drawing shapes. A region captured with `mss` can be converted into a `PIL.Image` object for further analysis or overlay. For example:

```

from PIL import Image, ImageDraw
import mss

with mss.mss() as sct:
    # Grab full monitor and convert to PIL
    screenshot = sct.grab(sct.monitors[1])
    img = Image.frombytes('RGB', screenshot.size, screenshot.rgb)

    # Define region coordinates (left, top, right, bottom)
    region_box = (100, 100, 600, 600)

    # Crop the region for analysis
    cropped = img.crop(region_box)

    # Draw an overlay rectangle on the original image
    overlay = img.copy()
    draw = ImageDraw.Draw(overlay)
    draw.rectangle(region_box, outline="red", width=3)

    # Save outputs
    cropped.save('cropped.png')
    overlay.save('overlay.png')

```

This demonstrates how to crop a region and draw a rectangle overlay using `ImageDraw.rectangle`. Region coordinates must be validated to fit within the image dimensions.

Phase 9 Implementation Plan

Goals and Acceptance Criteria

The audit document specifies the **named screen regions** feature with the following goals ¹:

- **Add region definitions to the configuration:** Under the top-level `regions` key, define named rectangles with `left`, `top`, `width` and `height` values. Example:

```
regions:
  full_screen:
    left: 0
    top: 0
    width: 1366
    height: 768
  top_left:
    left: 0
    top: 0
    width: 320
    height: 240
  dialog_box:
    left: 100
    top: 520
    width: 1160
    height: 180
```

- **New scripts:**
 - `scripts/list_regions.py` – list available region names for a profile or config.
 - `scripts/capture_region.py` – capture a named region and save a PNG plus JSON metadata (monitor index, region coordinates, timestamp).
 - `scripts/analyze_region.py` – reuse existing diagnostics (mean brightness, pixel change counts, etc.) on a named region.
- **Validation:** Region coordinates must be within the captured monitor; names must be unique. Invalid configurations should raise a clear error.
- **Backward compatibility:** Existing `--left`, `--top`, `--width`, `--height` CLI overrides should still work for ad-hoc cropping. Named regions provide convenience but do not replace manual coordinates ¹.
- **Exclusions:** Phase 9 does *not* include OCR, template matching or any decision logic ¹.

Proposed File Additions

File	Description
<code>src/perception/regions.py</code>	New module to load region definitions from config files and provide helper functions: <code>list_regions(config)</code> , <code>validate_region(name, rect, monitor)</code> , <code>grab_region(sct, monitor_index, rect)</code> and <code>crop_image(image, rect)</code> . It should depend only on existing perception code and <code>mss</code> / <code>Pillow</code> .
<code>scripts/list_regions.py</code>	CLI script that loads a config/profile, lists the region names and their coordinates, and optionally validates them against the detected monitor resolution.
<code>scripts/capture_region.py</code>	CLI script that loads the config/profile, selects a named region (or uses <code>--left/top/width/height</code>), captures the corresponding rectangle using <code>mss.mss().grab()</code> , and writes a PNG plus JSON metadata (region name, monitor index, coordinates, timestamp).
<code>scripts/analyze_region.py</code>	CLI script that captures a named region and feeds it into existing image diagnostics (from Phase 8) to compute statistics (min, max, mean, std deviation, changed pixels) and writes a JSON report.
<code>docs/REGIONS.md</code>	Documentation explaining how to define regions in <code>config/default.yaml</code> and profiles, how to calibrate coordinates, and examples of using the new scripts. It should stress that the region system is purely diagnostic and does not trigger any input.
<code>PHASE9_NOTES.md</code>	Brief notes describing what was added in Phase 9, how to test it, and any outstanding considerations.

Updating Configuration and Profiles

- `config/default.yaml`: Add a `regions` section with sensible defaults for RetroArch's main window size (e.g., `full_screen`, `menu_area`, etc.). These values may need to be calibrated by the user; provide examples only.
- `profiles/_template.yaml`: Add a commented `regions` section to illustrate how profile-specific regions can override or extend the defaults.
- **Existing profiles**: For the sample profile `retroarch_menu_test`, include at least one region (e.g., `menu_area`) with approximate coordinates. The docs should warn users to measure coordinates using the region capture tool.

Suggested Implementation Steps

1. **Set up a development environment** (if not already done):

```
git clone https://github.com/coltsfan0791/gameplay-automation-retroarch
cd gameplay-automation-retroarch
python -m pip install -r requirements.txt
```

1. **Create** `src/perception/regions.py` with functions to:
2. Load region definitions from the active config (`default.yaml` or the profile's config).
3. Validate that each region has `left`, `top`, `width`, `height` integers and fits within the monitor resolution.
4. Provide a helper `get_region(name)` that returns the rectangle dict and monitor index; fall back to CLI overrides.
5. **Write** `scripts/list_regions.py` using `argparse` to accept `--config` or `--profile`. It should print region names and coordinates, and optionally verify that they fit within `mss.mss().monitors[monitor_index]`.
6. **Write** `scripts/capture_region.py` using `argparse` to accept `--region` and optional `--monitor` overrides. Use `mss.mss().grab()` to capture the region and write a PNG to `logs/screenshots/region-YYYYMMDD-HHMMSS.png`. Save a JSON sidecar with metadata (profile, region name, coordinates, monitor index, timestamp).
7. **Write** `scripts/analyze_region.py` to reuse functions from Phase 8 for computing image statistics. Accept `--region` and `--monitor`. Output a JSON report with metrics (min/max pixel values, mean, standard deviation) and changed-pixel counts when comparing to a baseline image (feature to be added in Phase 12).
8. **Update** `README.md` with instructions on using the region scripts:
9. List regions: `python scripts/list_regions.py --profile retroarch_menu_test`.
10. Capture a region: `python scripts/capture_region.py --profile retroarch_menu_test --region dialog_box`.
11. Analyze a region:
`python scripts/analyze_region.py --profile retroarch_menu_test --region full_screen`.
12. **Test locally** (without pressing any inputs) by running the above commands. Confirm that region PNGs and JSON metadata appear in `logs/screenshots/` and that the CLI gracefully reports errors for invalid region names or out-of-bounds coordinates.

Verification Checklist

- [] Region definitions are added to `config/default.yaml` and relevant profiles.
- [] `src/perception/regions.py` can list, fetch and validate regions.
- [] New scripts (`list_regions.py` , `capture_region.py` , `analyze_region.py`) function correctly with informative help messages.
- [] Capture script saves a PNG and a JSON sidecar containing profile, region, coordinates, monitor index and timestamp.
- [] Analyze script computes statistics on the cropped region using existing diagnostic code.
- [] Existing CLI overrides (`--left` , `--top` , `--width` , `--height`) still work and bypass named regions.
- [] Documentation (`docs/REGIONS.md` , `README.md`) is clear, with examples and safety warnings.

Conclusion

The RetroArch gameplay automation project has established a robust foundation for scripted playback and diagnostic perception. The next step, **Phase 9**, introduces a named region system that allows users to define and reuse screen areas without hardcoding coordinates in command lines. By leveraging `mss` for fast region capture ²⁵ and `Pillow` for cropping and overlays, the project can implement region listing, capture and analysis scripts that maintain the safety-first philosophy of earlier phases. This phase lays the groundwork for more sophisticated vision-driven features (template matching, conditional gates, state machines) in later phases while keeping autonomy disabled until diagnostics are mature.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 [raw.githubusercontent.com](https://raw.githubusercontent.com/coltsfan0791/gameplay-automation-retroarch/main/docs/AUDIT_AND_ROADMAP.md)
https://raw.githubusercontent.com/coltsfan0791/gameplay-automation-retroarch/main/docs/AUDIT_AND_ROADMAP.md

23 24 25 [How to Take Screenshots with Python MSS](https://screenshotone.com/blog/mss-python-screen-capture/)
<https://screenshotone.com/blog/mss-python-screen-capture/>